Computer Science & IT

Data Structure & Programming



Linked List

Lecture No. 01



By- Abhishek Sir

Recap of Previous Lecture



Topic

Topic

Topic

Topic

Topic

Circular queue

queue using stack (2 stack)

Slide

Topics to be Covered



- Topic
- Topic
- Topic
- Topic
- Topic

Basic of pointer & structure

Dynamic Memory allocation

Slide



Topic : Deque & Priority Queue

Deque : Doubly ended queue

Four operation : Insert rear

Delete rear

Delete front

Insert front

Slide



Topic : Deque & Priority Queue

priority queue : priority queue is a queue in which element can inserted in any order but deletion will be based on priority.

e.g. Max Heap, Min Heap } Tree

Slide



Topic : Pointer & Structure

int a; a is a integer variable

a = 5;



Address of a is 1000 (Assume)

1000 ← Address of a integer variable

pointer variable

pointer variable are capable of Holding address of another variable of Similar Data type.

Slide



Topic : Pointer & Structure

Declaration : `int *ptr;` ptr is a pointer to an integer

Initialize : `int *ptr = &a;`

`&` Address of

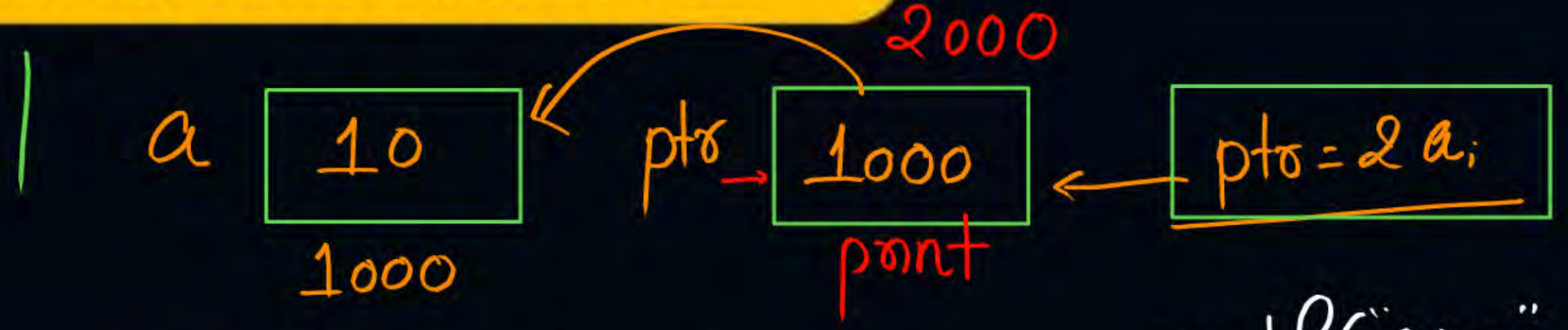
`*` dereference operator

Assignment : `ptr = &a;`

Slide



Topic : Pointer & Structure



printf("%d", a); = 10

printf("%u", &a); : 1000

printf("%u", ptr); : 1000

print("%u", *ptr); 10

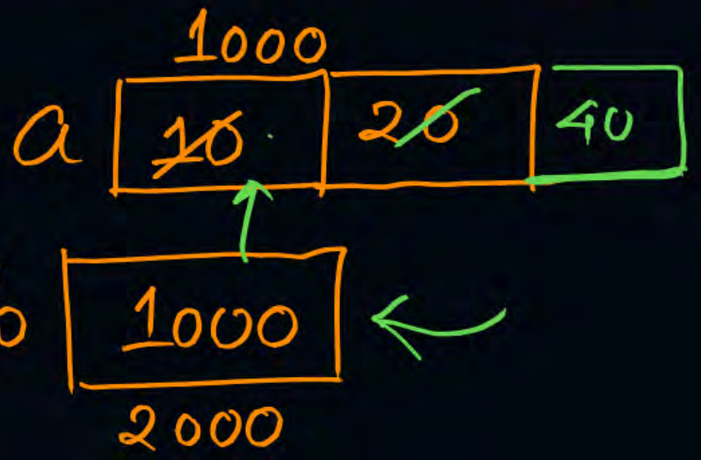
printf("%u", &ptr);

2000

Slide

Topic : Pointer & Structure

```
#include<stdio.h>
int main (){
        int a, *b;
        a = 10;
        b = &a;
        a =a+10;
        *b = *b+20;
        printf("%d", a);
        return 0;
}
```



Declaration

*b → 20+20 → 40

Slide



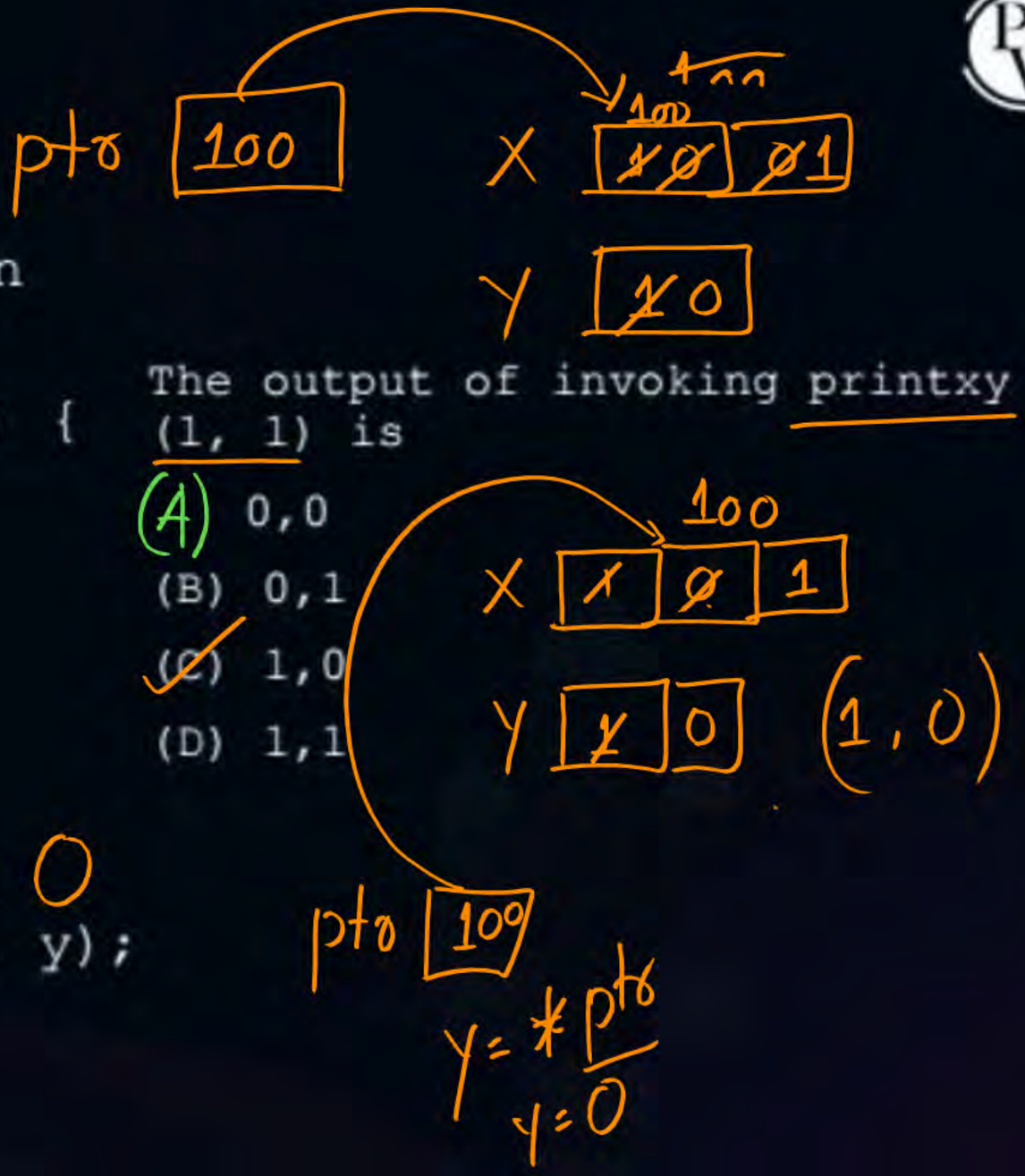
GATE 2017

#Q Consider the following function implemented in C:

```
void printxy (int x, int y) {
        int *ptr ;
        x = 0;
        ptr = &x;
        y = * ptr;
        * ptr = 1;
        printf ("%d, %d," x, y);
}
```

The output of invoking printxy (1, 1) is

(A) 0,0
(B) 0,1
(C) 1,0
(D) 1,1



Slide



GATE 2017

#Q Consider the following function implemented in C:

```
void printxy (int x, int y) {
        int *ptr ;
        x = 0;
        ptr = &x;
        y = * ptr;
        * ptr = 1;
        printf ("%d, %d," x, y);
}
```

Slide



Call by reference

```
# include<stdio.h>
void fun( int*, int*)
int main() {
    int a=10, b=20;
    fun (&a, &b);
    printf("%d", a);
    printf("%d", b);
}
```

prototype

a 100 | 1030

b 200 | 2050

200 ↑

```
void fun( int *ptr1, int*ptr2){
    *ptr1 = *ptr1 +20;
    *ptr2 = *ptr2 +30;
}
```

ptr1 100

ptr2 200

10

20+30

jump

Slide



Topic : Pointer & Structure

Structure

Array is collection similar Data type

Structure is collechon of dissimilar Data type

```
struct example {
    int a;
    char ch;
};
```

→ Name of structure

} member

Slide



Topic : Dynamic Memory Allocation

Structure variable

struct example a_1;

a_1 is a structure variable

Static allocation

Compiler time

To access member "." Dot operator is used

$a_1.a = 10;$

$a_1.ch = 'a';$

Slide



Topic : Dynamic Memory Allocation

```
Stouct example a1;
Stouct example *ptr;
ptr = &a1;
```

← pointer to structure

To access Member of structure using pointer we use "→" operator

```
ptr→ a = 10;
ptr→ch = 'a';
```

Slide



Topic : Dynamic Memory Allocation

* Array : Compile time allocation
* Extra space required at run time then array is Not useful.
* Linked list memory allocation & Deallocation Done Run time
* During run time memory allocation Done

Slide



Topic : Dynamic Memory Allocation

```
Struct example{
    int a;
    char ch;
};
```

malloc() ← memory allocation

Calloc() ← Contiguous allocation

```
malloc ( sizeof(struct example));
```

malloc (address) allocates memory & return pointer [void]

Slide



Topic : Dynamic Memory Allocation

```
Struct example *ptr;
ptr = (struct example*) malloc( sizeof( struct example));
```

(struct example*) → void pointer converted to Structure pointer.

Not mandetory in C.

```
ptr = malloc( sizeof(struct example))
```

Correct Syntax

Slide



Topic : Dynamic Memory Allocation

```
ptr->a = 10;
ptr->ch = 'a';
```

free or deallocate the memory: free(ptr);

↑ Deallocate

if Not free / Deallocated then it leads to Memory leak;

Slide



Topic : Dynamic Memory Allocation

```
struct example {
    int a;
    char ch;
};

struct example a1;
struct example *ptr;
```

Equivalent

Rename

```
typedef struct example {
    int a;
    char ch;
} Exmp;

Exmp a1;
Exmp *pt;
```

typedef Creates lias : another Common name

Slide

Q. Suppose a stack implementation supports an instruction REVERSE, which reverses the order of elements on the stack, in addition to the PUSH and POP instructions. Which one of the following statements is TRUE with respect to this modified stack?

(GATE_2014_2 M)

(A) A queue cannot be implemented using this stack.

(B) A queue can be implemented where ENQUEUE takes a single instruction and DEQUEUE takes a sequence of two instructions.

(C) A queue can be implemented where ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction.

(D) A queue can be implemented where both ENQUEUE and DEQUEUE take a single instruction



2 mins Summary

Topic — pointer Basic

Topic — Call by reference Basic

Topic — Structure, "." & "→" operator

Topic — Dynamic allocation of Memory

Topic

Slide

THANK - YOU

Slide